Senior Capstone Proposal – 25

Title	Remote Eye-Tracking UX Testing Software

Sponsor	Company name: PFW CS Department Faculty Advisor: Prof. Jay Johns
Type	☒ Application Development ☐ Research-focused ☐ Information Systems

Description	Develop an innovative software solution that enables remote eye-tracking during user experience (UX) testing. This software will utilize the built-in camera on users' personal computers, allowing us to capture and analyze eye movement data remotely during User Experience testing. By integrating advanced algorithms, the software will process the video feed from the user's camera to track eye movements and provide quantitative data on where users focus their attention during interactions with digital products. **Key Features:** **1.** Camera-Based Eye Tracking: The software will use the users' computer or phone cameras to track eye movement, eliminating the need for specialized eye-tracking hardware. **2.** Remote Testing Capability: UX testers can participate from anywhere, providing flexibility and increasing the diversity of testing environments. **3.** Data Collection and Analysis: The software will collect quantitative data on user focus points, gaze paths, and time spent on various elements, enabling detailed analysis of user behavior. **4.** User Privacy and Security: All data will be securely encrypted, and user privacy will be a priority, with clear consent protocols and data anonymization. **5.** Real-Time and Post-Session Reporting: The software will offer realtime feedback during testing sessions and detailed post-session reports to facilitate deep insights into user interactions. **Goals:** • Enhance Remote UX Testing: Provide a robust tool that allows for comprehensive UX testing without the geographical constraints of a physical lab. • Improve Data Accuracy: By capturing real-time eye movement data, we aim to gain more precise insights into user behavior and preferences. • Increase Accessibility: Make high-quality UX testing available to a broader audience by using widely available hardware (webcams) and eliminating the need for testers to visit a specific location. **Target Audience:** • UX Researchers and Designers looking for more detailed insights into user behavior. • Product Managers seeking to improve digital product usability based on quantifiable data. • Companies looking to conduct large-scale remote UX testing. • Students enrolled at PFW in Software development or User Experience courses looking to gain feedback from users. **Expected Impact:** This software will revolutionize how UX testing is conducted by removing physical barriers and providing detailed, actionable data. It will empower
	teams to make more informed design decisions based on real user interactions, ultimately leading to better user experiences across digital platforms.

Team size	☐ 2 ☐ 3 ☒ 4 ☒ >4
Required backgrounds	A strong motivation to work on a project that requires front end coding, back-end coding, and data-analysis.
Required resources (HW/SW)	There is no "required" programming language but the database will be stored on a local server rather than AWS or Google Firebase. Here is a suggested development stack to use with this project but the team is encouraged to determine the best tools for this project. **Frontend Development:** - JavaScript/TypeScript: Essential for building interactive, real-time interfaces. TypeScript is particularly recommended for its type safety, which helps in maintaining a large codebase. - React.js: A popular JavaScript library for building user interfaces. React's component-based architecture is well-suited for developing complex UIs. - WebRTC: A set of APIs that enables real-time communication directly from web browsers, crucial for handling video streams from the webcam. **Backend Development:** - Python: Python is highly recommended for its vast ecosystem of libraries related to computer vision and data analysis. It's easy to write and maintain, making it suitable for rapid development. - Django or Flask: Both are robust frameworks for building scalable web applications. Django is more feature-complete with built-in authentication, ORM, etc., while Flask offers more flexibility with a lighter footprint. - OpenCV: An open-source computer vision library in Python that can be used for image processing and eye-tracking algorithms. - TensorFlow or PyTorch: These deep learning frameworks could be employed if the project requires advanced computer vision tasks, such as enhancing the accuracy of eye-tracking or processing large datasets. **Computer Vision and Eye-Tracking:** - Dlib or Mediapipe: These libraries offer pre-built models for facial landmarks detection, which is essential for accurate eye-tracking. Mediapipe, developed by Google, is particularly efficient and optimized for real-time applications. **Data Storage:** - PostgreSQL: A powerful, open-source relational database that handles structured data well, suitable for storing user data, test results, etc. - MongoDB: An alternative NoSQL database option, ideal if the data collected is more varied or unstructured. **Data Analysis and Reporting:**

	- Pandas/Numpy: Python libraries that are industry standards for data manipulation and analysis. - Matplotlib/Seaborn: Libraries for creating detailed visual reports from the collected data. - Jupyter Notebooks: For creating shareable analysis reports and running exploratory data analysis (EDA). **Deployment:** - Docker: For containerizing the application, making it easy to deploy across different environments. - Kubernetes: For orchestrating containerized applications, ensuring they run reliably when deployed across a cluster of machines. **Security and Compliance:** - SSL/TLS: For securing data in transit. - OAuth2: For managing secure user authentication and authorization. **Additional Tools:** - WebAssembly (Wasm): If you need high-performance computing directly in the browser (e.g., for real-time eye-tracking), compiling parts of the code (like critical sections of the OpenCV library) to WebAssembly can significantly enhance performance. - Why this Stack? - Scalability: The stack is designed to handle potentially large amounts of data and users. - Performance: Real-time video processing and eye-tracking require efficient handling of video streams and image processing, which this stack supports. - Flexibility: The combination of Python for backend and data processing, along with JavaScript/React for the frontend, allows you to build a fullstack solution that's both powerful and user-friendly. - Community and Support: The chosen technologies have strong community support, meaning more resources and libraries are available, and any issues are likely to have existing solutions. This stack should provide a solid foundation for building a robust and scalable eye-tracking software solution that meets your project requirements.
Technology Disclosed? If so, what?	
NDA or IP Assignment agreement requested?	No

Other notes	